

Three classes of scale failure

Three classes of scale failure, and one that runs backwards

Scale testing a production PostgreSQL system I built alone

Draft 5 — 17 August 2026. §5 describes a plan, not an execution.

Summary

I put a production database under roughly a hundred times its real load and looked for the points where it would stop behaving the way it behaves today. Three turned up, different in kind rather than degree — each detected by a different instrument, each remedied by a different change.

Memory. A bulk-read path spilled 1,437 MB to temporary files. The obvious remedy — enlarge the shared buffer pool — cannot work, because large scans deliberately bypass that pool. Along the way: the fastest configuration measured had the worst cache hit ratio and the most physical I/O.

Statistics. 7.6× degradation with no memory pressure at all. Detected by comparing estimated to actual row counts; remedied by `ANALYZE`; invisible to any instrument watching buffers.

Planner boundary, and this one runs backwards. A parallel-only plan becomes available when a table crosses 8 MB, so **the system gets faster as it grows across that threshold and the dangerous region is below the cliff, not above it**. Plan shape flips discontinuously, and production sits on the slower side today.

Two indexes were selected to ship, one resolving a 98.8% read improvement and a 93.85% reduction in cascade-delete cost. A third was measured and deliberately left unranked, because the evidence that would judge it cannot be collected single-connection. A fourth was registered expecting it to fail, and did.

The measurement cost more than the fixes. Eight instrument designs failed before one worked, and four produced plausible numbers that were wrong — three reported a *saving* on an index insert, which is physically impossible. One passed three registered gates and five negative tests, then was falsified by a null distribution I built for a statistic I had assumed did not need one. Three sessions were lost to a gate rather than a protocol: the fix was deleting an assertion.

One thing up front. Production holds 238 completed workouts; the mirror holds 28,053. Every figure here was measured at mirror scale, and none of the improvements will be visible in production latency today. These are predictions about a system growing into them. §6 sets out the rest.

1. Why this exists

The application is one I built and run: a fitness tracker with real users, live, maintained solo since January 2026. TypeScript throughout, React front end, Node and Express back end, PostgreSQL 16 through the raw driver with no ORM.

The study is deliberate. Job postings in the roles I want ask consistently for experience with production systems under load. I have not held a commercial engineering title, so rather than claim the experience I generated it — against a system where I already knew every design decision, having made all of them.

That has an obvious weakness and I would rather name it than have it noticed. **One person studying their own system has no reviewer.** Everything in §3 about pre-registration, null distributions and predictions written so they can fail is a substitute for that, and a partial one.

What it buys is real: total access to the system and its history, no institutional constraint on breaking things, and freedom to spend four sessions establishing that an instrument does not work.

The deliverable was always the findings, not a faster application. That was set at the start, and it changed what the work optimised for. The application did get faster; that is the least interesting sentence here.

2. Three classes of scale failure

The mirror held 1,400 users, 28,053 workouts and 382,919 sets. Three failure points turned up, and the taxonomy is operational rather than descriptive — knowing which class you are looking at tells you what to reach for.

class	found	detected by	remedied by
Memory	1,437 MB spill on a bulk read	Buffer accounting	Configuration
Statistics	7.6× degradation, no memory involved	Estimated vs actual rows	ANALYZE
Planner boundary	An 8 MB threshold flipping plan shape	Plan shape across scales	An index, or a threshold

2.1 Memory: the ring buffer, and why more cache does not help

A bulk read spilled 1,437 MB to temporary files at `work_mem = 4MB`. Sorting more than fits in memory is the oldest problem in databases and the remedy is unglamorous.

The interesting part is the remedy that does *not* work. The instinctive fix for heavy I/O is raising `shared_buffers`. That was registered as an intervention with a prediction attached: **it will not help**. It did not, and the reason is structural. A large sequential scan does not pull the table into the shared pool — it uses a **ring buffer**, a small bounded circular allocation, so one scan cannot evict everything else. The design is correct, and it means a scan-heavy workload is largely insensitive to the size of the pool it is bypassing.

This produced the study’s most counterintuitive result, and it is a fact about Postgres rather than about this application: **blks_read does not mean “read from disk.”** It means “not found in `shared_buffers`” — and such a page may be sitting in the OS page cache, which is memory, and fast.

The fastest configuration measured had the worst cache hit ratio and the highest blks_read. It was fast because the ring buffer was doing its job and the page cache absorbed the reads. A team monitoring hit ratio as a health metric would have flagged the healthy configuration as the sick one — and would have caught none of the three failures below, each of which is invisible to the instrument that finds the others.

2.2 Statistics: degradation with no memory involved

The planner chooses access paths by estimating row counts, and those estimates come from `ANALYZE`. Stale statistics mean confident decisions on a false picture — a nested loop where a hash join was right, an index scan where a sequential scan would have won.

Measured: **7.6× degradation**, no spill, no buffer starvation. The detecting instrument is not buffer accounting but the ratio of estimated to actual rows, which is visible in any `EXPLAIN ANALYZE` output and routinely ignored. The remedy is nearly free. The difficulty is entirely in noticing.

This class also surfaced a defect in the study’s own apparatus, which is the same failure in miniature. The data generator built the tables, ran `ANALYZE`, and then derived one more table — which therefore carried statistics describing a state that no longer existed: **99,456 estimated rows against an actual 67,990, a 46% inflation**. It was caught by a fingerprint check, not by anyone looking. A study about stale statistics was, for a while, measuring against stale statistics.

2.3 Planner boundary: the cliff that runs backwards

Postgres considers a parallel sequential scan only when the table exceeds `min_parallel_table_scan_size`, 8 MB by default. Below that the parallel plan is not rejected on cost — **it is not generated at all**.

For the path examined here, the efficient plan — sorting below the join rather than above it — **exists only as a parallel plan**. So below the threshold the planner cannot reach it. Above, the plan becomes available, is cheaper, and wins.

The system therefore gets faster as the table crosses 8 MB, and the dangerous region is below the cliff. Growth fixes the problem, and the sluggish region is the one you are in on the way up.

Two further properties. **It is a threshold, not a curve** — plan shape flips at a boundary, with no gradual worsening to monitor and no leading indicator in a latency graph. And **production is on the sort-above plan today**, established by capturing its plans rather than inferred. The *cost* of that is latent, because the table is small enough that neither plan takes long. Both halves have to be stated together.

This is the class needing an instrument nobody routinely uses. Buffer accounting will not find it; row estimates will not find it. Only **capturing plan shape across a range of table sizes** finds it — which requires deciding in advance that plan shape is worth recording, before any evidence that it changes.

3. How I know these numbers are real

The interventions in §4 are, on their face, unremarkable. An index on an unindexed foreign key made a query 98.8% faster. That is the kind of result anyone can produce, and the kind nobody should believe on sight.

Eight instrument designs failed before one worked. Four produced plausible numbers that were wrong. Every one of those four was wrong in the direction that flattered the intervention, and none was caught by the tests written to catch exactly that.

3.1 The problem

The target was the insert cost of an index: how much extra work does it create when a row is added?

The expected answer is small — a B-tree leaf insert is among the cheapest things a database does. The environment was a laptop: a virtualised Linux VM under Docker Desktop, sharing a CPU with a browser and an editor. The signal was around 0.016 ms per operation. **The measured noise floor was larger**.

The standard answer is to measure many times and take a robust statistic. That answer is wrong here, and establishing why took five designs.

3.2 Four layers, each catching what the previous could not

Negative tests prove an instrument does what it says. Remove the thing you are measuring, confirm the instrument reports zero. Five passed. They proved the harness could distinguish index-present from index-absent, that a rollback left nothing behind, and that the arithmetic was arithmetic. They did not catch that the arms were being compared in an order that biased the result.

A diagnostic at scale proves that what it says is enough. A six-victim pass — 0.8 seconds of runtime — caught what all five negative tests missed. The instrument worked correctly and measured the wrong thing, and only realistic data made that visible.

Only a null distribution proves the number means anything. A design had passed three registered gates and five negative tests and produced a clean figure. I ran the same instrument with the index never created — a sham, where the correct answer is exactly zero. **It returned −0.0485 ms, an 18.24% effect, from a treatment that did not exist.** Everything above had been computed against a statistic whose null was assumed to be zero and never checked.

And none of it helps if the statement under test is not the statement production runs. Two sessions later, reading the harness against the application source, I found the benchmark’s SQL had silently diverged: a plain `INSERT` where production runs an upsert. Five negative tests had passed against it, each measuring the wrong thing correctly.

The fourth layer is the one I did not anticipate. The first three ask whether the instrument is sound. The fourth asks whether it is pointed at the right thing, and no amount of instrument validation reaches it.

3.3 The eight failures

#	design	what happened
1	Per-victim warmup	Reported the index-present arm 3.8× faster . An index cannot make an insert faster.
2	One whole-set warm pass	Failed its floor gate at 22.77% / 30.34%.
3	Eight saturating passes	A real plateau was found — pass 1 sat +360% above it — and the sign failure got <i>worse</i> .
4	ABAB counterbalancing	Passed all three gates. Falsified by its own sham-null at −18.24%.
5	ABBA counterbalancing	Failed its centring gate. Registered hard stop taken.
6	Warm pass + FPI suppression	Two gates failed. Aborted inserts extend the relation, so a warm pass cannot warm pages that do not exist yet.
7	<code>VACUUM</code> reset, three variants	The registered gate failed three times, three distinct mechanisms.
8	No reset at all (a registered pilot)	Failed. The null came back −32 records where zero was registered.

Designs 1 through 3 share a property worth stating plainly: **each reported a saving on an index insert, and each ran in the direction that made the intervention look better.** That is not coincidence. When contamination correlates with execution order and the treatment arm runs second, the contamination loads onto the treatment.

Design 4 is the one I would have shipped. It passed everything it was asked to pass.

3.4 The diagnosis that took eight arms to see

Designs 4 and 5 failed for what looked like drift across execution order. I characterised it as a monotone drift and designed against that. **That characterisation was wrong, and it drove two designs.**

Running eight arms instead of four made the shape visible. **Seven of eight arm positions sat within ±4.5% of each other. Position five alone was 69% high — and high for every victim simultaneously.**

That is not drift. Drift is smooth and cumulative and can be cancelled by pairing arms symmetrically. This is an episodic excursion: one arm contaminated as a whole. **No fixed arm ordering cancels it**, so designs 4 and 5 were attempts at a class of solution that could not work. The claim that adjacent pairing cancels a monotone drift, which I had asserted and built on, is also simply false. I had not proved it. It sounded right.

3.5 The fix that was a deletion

By session seven the modality had changed: timing abandoned for WAL accounting — counting write-ahead-log records, a structural quantity, deterministic by construction and immune to scheduler noise in a way no timing can be.

The instrument needed to return the database to a known physical state between arms, because aborted inserts leave dead tuples and extend the relation. I specified `VACUUM` as the reset and registered a gate: relation size must return to its pre-arm value, asserted.

It failed. I amended the registration — before the run, in the direction that made the gate harder — and it failed differently. A third variant failed a third way:

- **Truncation threshold.** `VACUUM` attempts truncation only with at least `rel_pages/16` freeable trailing pages. That was 78. Three were free.
- **Index-scan bypass.** Under 2% of pages dirty, `VACUUM` skips the index scan, leaving 193 dead line pointers.
- **Non-idempotence.** A second bare `VACUUM`, with no intervening write, moved the size again.

The obvious next move is a fourth reset design. I stopped instead, on a rule set in advance: two failures of one mechanism is a design question, three is a modality question, and re-engineering a reset until one passes is tuning the instrument against its own gate.

The diagnosis was that **VACUUM has two fixed points on relation size, three pages apart** — 1237 after a bare vacuum, 1240 after a pass-then-vacuum, each stable across repeated readings. The arm protocol necessarily alternates between both operations, so the relation oscillates and an exact-equality gate can never be satisfied.

The next session ran the same **VACUUM** with the assertion removed. **It passed completely** — every arm exactly integral, the null exactly zero, no gate widened.

The VACUUM was never the defect. Asserting relation size on top of it was. Across all six snapshots of the successful run, relation size sat at 1257 pages, perfectly stable and simply not asserted. Three sessions were spent on a gate, not a protocol, and the fix was deleting a line.

The generalisation transfers: **a gate can be on a proxy, and the proxy can have properties the target does not.** The property I cared about was that the measured delta was uncontaminated by physical drift. Relation-size equality was a convenient stand-in, and **VACUUM**’s hysteresis belongs to the stand-in alone. I gated on it because it was easy to assert.

3.6 Two instruments that disagree

With WAL records as the statistic I had two independent readings: **EXPLAIN (ANALYZE, BUFFERS, WAL)**, which reports per-statement counters, and **pg_waldump**, which decodes the record stream.

They do not agree. The decode shows **three foreign-key-check Heap/LOCK records per statement that the EXPLAIN counter does not attribute.** Both are correct about what they measure.

The consequence is specific: **an absolute figure from either instrument is wrong, and wrong by a different amount depending on which one you read.** The three lock records appear in both arms of a paired comparison and cancel in the difference. Everything this study reports about index cost is a **delta between arms**, never an absolute from one. I had been enforcing that on principle — it had to be corrected into two successive session briefs where I had drafted a prediction as an absolute. Two instruments disagreeing on absolutes and agreeing on deltas turned principle into evidence.

The decode also settled a mechanism, more precisely than I had predicted. The upsert costs one more WAL record per row than the plain insert, and the record is **XLOG_HEAP_CONFIRM** — but the heap insert itself carries **flags: 0x04 (XLH_INSERT_IS_SPECULATIVE)** where the plain form carries **0x00**. I had predicted a record *following* the insert. The insert is *marked*, and the confirm record completes a mechanism rather than trailing one.

3.7 When a modality cannot answer a question

The last measurement needed was whether the second index breaks HOT (heap-only tuple) updates. A HOT update is nearly free: when an update modifies no indexed column, Postgres skips every index. Breaking HOT would make the index expensive on the hottest path in the application.

The instrument measured HOT status through **pg_stat_user_tables**, and rolled back every transaction by design so state was unchanged between arms. **Those two facts are incompatible.** A subtransaction abort discards the pending statistics counts, so the counter reads zero regardless.

The prediction returned VOID rather than zero because I had written it with a void condition — a registered check that fires when the measurement cannot be established. **Zero and “not measurable” are indistinguishable in the output and completely different in meaning.** Zero was also the predicted result. Had the prediction been written to return a number, it would have returned zero, and I would have recorded a confirmation of a hypothesis I had not tested.

This is not an unanswered question but an unanswerable one **for this modality**: no arrangement of a rollback-based instrument can establish HOT status for the statement it measures.

The resolution was not a better instrument. It was reading the schema. HOT breaks when an update modifies a column appearing in some index — so I listed every indexed column across all sixteen indexes on the table. The updated column appears in none, and the new index’s three columns are each already indexed elsewhere. **It adds no new column to the indexed set, so it cannot break HOT that the existing set does not already break.** Structural, not empirical.

The error worth recording is that I designed a measurement before checking whether the schema already answered the question. It cost nothing, because the check was two queries. The ordering was still wrong.

3.8 A diagnostic that carries no information at production scale

Postgres tracks how many times the planner has chosen each index. Sixteen indexes on the table in question, counters ranging from 98,339 scans down to zero. Four had never been used. That reads like hard evidence.

Production holds 238 live rows in that table. Below a few hundred rows the planner reads the whole table more cheaply than it walks an index, whatever queries exist. The counters do not measure whether an index is needed; they measure whether the table is big enough for the planner to bother.

The proof is in the data. One index supports a query that runs on **every completed workout** — in the post-workout path, traced through the code to its route. **Its scan count is 1.** Had I retired indexes on scan counts, I would have dropped one serving a hot path.

The counters were good for narrowing: they selected which five indexes to investigate. Every verdict came from tracing query shapes through the codebase — finding, for instance, that the one dynamic query builder capable of emitting a given shape has no caller anywhere.

This generalises the study’s central finding. The reason the cliffs in §2 are latent is that production is small. That same smallness makes production’s own statistics unable to tell you anything about them. **The instrument that works at 28,053 rows is not the instrument that works at 238, and the failure is silent in both directions.**

3.9 Scoring the pre-registration

A pre-registration that is never scored proves nothing — it sits in version control looking like discipline. Scoring was done deliberately last, after every measurement, so no prediction could be softened while its result was still forming.

Three hit, one partial, one missed, three never run, one void, and **three ambiguously worded in ways that mattered**. Four results are worse than the study’s own narrative suggests.

A rule whose two readings select different answers. The rule deciding between the two competing index fixes is ambiguous. Read as its text says — a delta — the arms sit 0.2 points apart and the rule’s own tie clause selects one. Read on the residual metric resolved shortly after registration, they sit 24.6% apart and it selects the other. The resolution predates the data and is the better metric. **But the registration was never amended in writing, so scoring it against its own text returns the opposite arm.** Both readings reported, neither reconciled. **A registered rule whose metric is resolved after registration must be amended in writing, or it scores against its original text.**

A hit whose scope missed where the harm landed. One prediction scored a clean HIT — and the intervention it endorsed regresses four cheap cells on the same query path by 20.7% to 32.1%. No registered prediction covered them. **An intervention can win every registered comparison and still be worse somewhere nobody registered.**

A hit whose rationale is refuted by its own confirming evidence. Another prediction held, and the plan capture demanded to confirm it shows the stated mechanism was wrong — the loop count I predicted would change was unchanged. Right answer, wrong reason, caught only because the registration demanded a capture rather than a number.

A void that cannot be redeemed. The one prediction that would have tested the write-side argument for the shipped intervention is void and unanswerable by this instrument family (§3.7). The argument stands on schema reasoning rather than measurement, and the scoring is where that becomes visible rather than assumed.

Three predictions were never run, scored NOT RUN with reasons rather than omitted. A pre-registration is only evidence of discipline if the unrun items are visible.

4. The interventions

Five were registered before any was measured, each with a prediction and a decision rule. Every figure is a **paired delta measured within a single session**, comparing the system against itself with the intervention present and absent and back again. Nothing is compared against a number recorded on a previous day; §3 explains why.

4.1 A — an unindexed foreign key

Ten foreign keys reference the workouts table. Exactly one is both non-empty and unindexed. A is a single-column B-tree index on it.

Read: −98.8%, and flat at OFFSET 0, 200 and 2000, with 0.1–0.3% drift between repeats. The plan capture shows the mechanism: a sequential scan costing 0.00..2086.88 and taking 1.141–1.246 ms becomes an index-only scan costing 0.29..8.43 and taking 0.001 ms, with `loops=1314` unchanged. The node’s share of plan cost falls from **92.5% to 4.7%**.

That unchanged `loops` figure explains why this result behaves differently from the others. **The other interventions remove loops; A removes the cost per loop.** Reducing iterations helps most when there are many, so its benefit decays as you page deeper. A makes each iteration nearly free, so its benefit does not decay. Flatness across offsets is the signature of the mechanism.

Cascade DELETE: −93.85% at the median, worst of thirty victims still 88.69%. Deleting a workout cascades to that table, and without the index each cascade scans it whole. Quoted against a floor the instrument measured for itself — the same comparison with no intervention gave 1.99% median, 6.72% at p95, 7.80% worst — so the effect is an order of magnitude above the instrument’s own noise. Attribution was audited rather than inferred: the other nine keys are empty or already indexed.

Insert: 1.0794 WAL records per row. A paired delta through WAL accounting, against a null that came back exactly zero.

One point of care. Against a plain `INSERT` the answer is exactly 1.0000 across all eighteen cells — a clean result, and the control confirming the mechanism: one extra B-tree leaf insert emits one extra WAL record. **But the application does not run a plain `INSERT`.** Against the real upsert the figure is 1.0794 — twelve of eighteen cells exact, fifteen excess records in five cells, of which an enumerated page-split term accounts for one. **The number reported is 1.0794.** Reporting 1.0000 would be reporting the control, and this study already made that error once (§3.2). The residual’s mechanism is not claimed.

4.2 B and E — one defect, two fixes, and a reversal

The query orders by `workout_date DESC NULLS LAST`. The index that would serve it provides `DESC NULLS FIRST`. Different orderings, so the index cannot satisfy the sort and the planner sorts itself.

B changes the index to match the query — a migration, a rollback plan, an additional index on a table already carrying sixteen, and maintenance forever. **E changes the query to match the index** — a query change in a normal sprint, nothing to roll back.

B won on latency: 19.37 ms residual against E’s 24.14 ms, from the disadvantaged first position. **E collapsed between-repeat variance from 57.5% to 0.5%** — not just faster but far more predictable, which for a user-facing path is arguably the more valuable property.

Two qualifications, both from the scoring (§3.9). The registered rule is ambiguous, and its two readings select different arms — 0.2 points apart on the rule’s plain text, which fires its tie clause for E; 24.6% apart on the resolved residual metric, which selects B. And **B regresses four cheap cells on the same path by 20.7% to 32.1%**, which no registered prediction covered. **B is not uniformly**

better than the status quo on reads; it is better on the cells the rule measured.

Why the winner did not immediately win. The rule weighed read latency alone, and winning it is not the same as being the right fix. For four days the position was that E should ship — zero deployment cost, around 99% of B’s first-page benefit, no permanent burden.

Two production facts reversed it. **The write mix:** 603 inserts against 3,932 updates, at a **93.8% HOT rate**. B’s cost is one WAL record per insert, and inserts are the rare operation. **And B’s HOT-neutrality is structural** (§3.7) rather than measured. Together, B’s entire cost falls on the rare path and none on the dominant one. A separate investigation then established that four of the sixteen indexes can be retired on code evidence, so B lands as the thirteenth on a rationalised table rather than the seventeenth on a cluttered one.

The ruling is B. Recorded as a reversal because it was one — a position held in the morning and changed in the evening on evidence not yet gathered when the position was formed.

4.3 D — the required negative result

D was registered expecting it to fail, and did. §2.1 explains why raising `shared_buffers` cannot help scan-heavy work. It is reported at equal prominence because a study that publishes only its successes is not reporting, it is advertising. It is also the most portable finding here: the reasoning applies to any Postgres system where someone is about to raise `shared_buffers` because a graph looks bad.

4.4 C — measured, and deliberately not ranked

C’s benefit is contingent on concurrent access, and the measurement environment is **single-connection by design** — which is what makes the timings reproducible. So the evidence that would judge C is, by construction, not collected. Ranking it would mean ranking on evidence that cannot speak to its main property. **The refusal is the finding.** It is recorded as measured, with its mechanism confirmed, and excluded from the ordering until a concurrency study exists.

5. Shipping it

A plan, not an execution.

Two indexes ship, as **online index creations against live traffic** — the system stays up throughout. That constraint is the point: creating an index the ordinary way takes a lock that blocks writes for the duration, which on a table with users mid-session is an outage. `CREATE INDEX CONCURRENTLY` builds in two passes while writes continue, at the cost of being slower and more fragile.

Three properties drive the runbook, each a way it goes wrong. **It cannot run inside a transaction block**, so tooling that wraps everything in one will fail confusingly. **A failed run leaves an invalid index behind** — visible in the catalogue, maintained on every write, never used by the planner, and requiring a drop before retrying. **The check is `pg_index.indisvalid` after each statement**, because nothing else will tell you. And it does two table passes plus a wait for concurrent transactions, so timing is recorded even though at this size it is seconds.

The retirement ships first and separately. Four indexes can be retired: three with no matching query shape anywhere in the codebase, and one that is a **byte-identical duplicate** of another under a different name. That duplicate is the clearest case — same columns, order and predicate; one chosen 324 times, the other never. Given identical alternatives the planner picks one and does not revisit, so **the unused twin can never be selected while its twin exists**, while still being maintained on every insert.

Those drops are separate from the index creations for three reasons. **Reversibility** — a failed create leaves something droppable; a bad drop leaves you rebuilding under load, which is the situation the exercise exists to avoid. **Blast radius** — three drops and two creates at once means that if something goes wrong, which thing went wrong is a question. **Attribution** — mixing an irreversible operation into a demonstration of safe online creation muddies what it demonstrates.

One dependency runs the other way: part of the argument for B is that it lands as the thirteenth index rather than the seventeenth. If the retirement does not land, the write-mix argument stands alone and is probably sufficient, but the ruling would be revisited rather than executed on an assumption.

The drops were not decided on usage statistics, for the reason in §3.8. Every verdict came from tracing query shapes through the source: does a matching query exist, and is it reachable — behind a route, a flag, a gate, or nothing. One index remains **unresolved rather than retired**: a matching query exists behind a live authenticated route, but the component that would call it is never rendered. That is a dead route and a dead component, which is a real finding, but not proof that nothing calls the endpoint.

None of this makes production measurably faster — every figure was measured at 28,053 workouts against production’s 238. The case for shipping now is that both indexes are cheap, both are structurally sound, and the alternative is applying them later under load, at exactly the moment when the system is degrading and the safe path is narrower. **The benefit is latent, and it is being paid for in advance.**

6. What this does not show

A study whose limitations are a footnote is asking to be taken on trust.

The scale gap. Production holds 238 live workouts and 765 rows in the table A indexes; the mirror holds 28,053 and 67,990. Every figure in §4 was measured at mirror scale and **none describes production today**. The findings are predictions about a system that grows into them, and if it never grows they never matter. But the cliffs are threshold effects (§2.3), so “not yet” and “never” are different, and the arrival point is knowable in advance.

The data is synthetic. Volume was generated, calibrated against measured production distributions — session counts, exercises per session, sets per exercise, the proportion of accounts never completing a workout. Calibrated is not real. **The distributions come from n=19 users, one holding 74% of all activity.** The shape of the tail is a declared modelling choice, not a measurement — it could not be a measurement, because the data does not exist. Where that shape matters to a finding, the finding is weaker than its number suggests.

The architecture differs. ARM64 in a container on a laptop against x86-64 on a VPS. **Only relative deltas within the container are claimed,** which is why the figures are percentages and paired differences rather than milliseconds.

Everything is single-connection. That is what makes the timings reproducible, and it means the study has **no concurrency evidence at all** — no lock contention, no pool exhaustion, no behaviour under simultaneous writers. C is unranked because of this (\$4.4), but the gap is broader: any finding here could behave differently under concurrent load.

Schema provenance — checked, and clean. The mirror is built from a snapshot generated from a local copy rather than production directly, and a dump-and-restore round trip changes how some constraints render. That mattered specifically: **CHECK constraints are evaluated per row on insert**, so a constraint present on production and absent from the mirror would mean an insert measurement did less work than the real one — instrument sound, answer wrong. Re-rendering is cosmetic; a difference in *presence* is not. It was checked rather than assumed: a production dump compared against a fresh regeneration, table by table, for the four tables every measurement touches. **No constraint, column or index exists on one side and not the other.** The only differences are two cosmetic cast renderings. The figures stand unqualified on this ground.

Constraints on two measured tables. Inserts into two of the four tables under measurement are evaluated against five CHECK constraints — four on one table, one on the other. They are present in every arm equally, so paired deltas are unaffected, and **no absolute per-row insert figure is reported for either table.** The table where insert cost was measured carries none. The constraint sets were confirmed identical across four catalogues — production, the mirror, the local copy and the schema snapshot — so the mirror enforces exactly what production enforces. Recorded because a limit that turns out not to bite is still a limit that was there.

Open items, listed rather than tidied away. The 15-record residual in A’s insert cost, of which a page-split term accounts for one. An index with no matching query shape anywhere that has a scan count of 1, left unexplained rather than rationalised. Why `VACUUM` is not idempotent on relation size — a visibility-map explanation is plausible and unverified. Why `EXPLAIN` omits the foreign-key lock records `pg_waldump` shows. One relation-size measurement that failed to reproduce in an early session and reproduced identically in the five sessions after it.

What the study is not. Not a benchmark — nothing compares Postgres to anything else. Not peer-reviewed, and one person’s work on one system; the pre-registrations and negative tests exist because there was no reviewer, and they are a partial substitute. Not finished — concurrency, replication and a CI regression gate are all outstanding.

7. Reproducing this

The repository is private, so this is written so nothing has to be taken on trust: every mechanism claimed above is a property of PostgreSQL, verifiable independently without access to this code.

Environment. PostgreSQL 16.11 in a container, 3,800 MB memory limit, 2 CPUs, `shared_buffers` 128 MB, `work_mem` 4 MB, `random_page_cost` 4, `default_statistics_target` 100, and `min_parallel_table_scan_size` at its 8 MB default. A verifier asserts forty values before any measurement runs and has itself been negative-tested — deliberately broken to confirm it fails.

The mechanisms, all independently checkable. The ring buffer’s bypass of `shared_buffers` and the meaning of `blks_read` (\$2.1). The parallel-plan gate (\$2.3). `VACUUM`’s truncation threshold at `rel_pages/16` and its index-scan bypass below 2% of pages dirty (\$3.5). `XLOG_HEAP_CONFIRM` and the `XLH_INSERT_IS_SPECULATIVE` flag (\$3.6). The discarding of pending statistics counts on subtransaction abort (\$3.7). None is specific to this application.

The method, which is the transferable part, and the whole of it. Pre-register the prediction, statistic, decision rule and stop condition before building the instrument — that is what makes a failed prediction a result rather than an embarrassment. Build a null distribution by running with the treatment absent and confirming it returns zero. Write at least one prediction that can return “not answerable”, because one that must produce a number will produce one. Register the direction a failure would take under a known mechanism, so an unexpected direction is reported as unidentified rather than explained afterwards. Commit amendments before the run they govern, and only when they make a gate harder — one that loosens a gate after seeing the data is not an amendment. Never widen a band to accommodate an instrument: removing a gate that is measuring the wrong thing, with the reasoning recorded, is a different act from relaxing one that is measuring the right thing inconveniently. Never edit a registered document in place; corrections are dated appends, because a study that quietly fixes its record cannot be audited. **And score the registration afterwards, including the predictions that were never run.**

Every one of the failures in §3 was caught by one of those rules rather than by care in the moment. The three I broke — the monotone-drift claim, the warm-pass protocol, the relation-size gate — were each a design property I believed because it sounded right and did not prove.

The evidence. Every session produced a pre-registration committed before its instrument existed, raw evidence files, and a session record including a section listing what was not verified. Those artefacts are dated and ordered in version control, which is what makes “registered before measuring” checkable rather than asserted. The repository can be opened to a named reviewer on request.

8. What I would check on a system I had not built

Everything above was found on a system where I knew every design decision, because I had made all of them. The transferable part is not the findings — it is the questions that produced them, and most of those questions can be asked on the first day of contact with a system.

Ask what the plan shape is, not just what the latency is. A latency graph cannot show a plan flip. When the planner switches strategy the change is discontinuous, and the graph shows a step with no explanation in it. `EXPLAIN` on the important queries, at the current data volume and at a multiple of it, will show a boundary that no amount of monitoring reveals. This is how the backwards cliff in §2.3 was found, and nothing else would have found it.

Do not trust the cache hit ratio. It is the most commonly dashboarded database metric and it can run the wrong way. In this study the fastest configuration had the worst ratio and the most apparent disk reads, because `blks_read` counts pages not found in the shared pool — which is not the same as pages read from disk. Before treating a low ratio as a problem, establish whether the workload is one that uses the pool at all.

Check that the benchmark runs the statement production runs. Mine did not. The harness had drifted to a simpler form of the query, and five negative tests passed against it — each measuring the wrong thing correctly. Diff the benchmark’s SQL against the application’s, by hand, before trusting any number it produces. No instrument validation catches this, because the instrument is working.

Check the size of a table before trusting its index statistics. Postgres records how many times each index has been chosen, and on a small table those counts measure the table’s size rather than the index’s necessity. Below a few hundred rows the planner reads the whole thing regardless. In this system an index supporting a query that runs on every completed workout had a scan count of one. Retiring indexes on those counts would have dropped one serving a hot path.

Look for indexes that are identical under different names. Two indexes here had the same columns, the same order and the same partial predicate, with different names. One had been chosen hundreds of times and the other never — because given identical alternatives the planner picks one and does not revisit. The unused one can never be selected while its twin exists, and is maintained on every write regardless. Comparing index definitions rather than index names finds these in a single query.

Read comments against the tests that cover them. The clearest example of this class came from a different corner of the same codebase, during unrelated work on authentication. A code comment described intended behaviour that a test three files away contradicted — correctly, and while passing. Both had been committed, both had been read, and the comment won because it was the artefact read first. The contradiction sat in the tree for months, and the real behaviour it concealed was that every signed-in user was being silently logged out once a week. **Prose has no failing state**, so a comment and a test can disagree indefinitely without anything going red.

Ask what the statistic’s null distribution is. Not what the number is — what the instrument returns when the thing being measured is absent. If nobody knows, nobody knows what the number means. This is the single most valuable question in the list and it took me five instrument designs to learn to ask it.

Ask what would have to be true for the measurement to be wrong, and then check that specific thing rather than gathering more of the same evidence. Three of the failures in §3 were found this way and none was found by measuring more carefully.

Appendix — terms

ANALYZE — the command that collects statistics about a table’s contents, so the planner can estimate how many rows a query will touch. Cheap; needs re-running as data changes.

B-tree — the default index structure. A sorted tree that lets the database find a row by value without reading the whole table.

blks_read — a counter of pages not found in the shared buffer pool. Commonly misread as “pages read from disk”; the page may be in the operating system’s cache, which is memory.

Cascade delete — when deleting a row automatically deletes rows in other tables that reference it. If the referencing column has no index, each cascade scans that table in full.

CHECK constraint — a rule the database enforces on every row as it is written. Evaluated per row on insert, so it has a cost.

EXPLAIN / EXPLAIN ANALYZE — asks the database to describe the plan it would use for a query; `ANALYZE` additionally runs it and reports what actually happened, including timings and row counts.

Foreign key — a column pointing at a row in another table. Not automatically indexed.

FPI (full-page image) — a complete copy of a page written into the log the first time that page changes after a checkpoint, so the page can be reconstructed after a crash. Makes log volume lumpy and checkpoint-dependent.

HOT update (heap-only tuple) — an update that modifies no indexed column, so the database can skip updating every index. Substantially cheaper than a normal update.

Index-only scan — reading data straight from an index without touching the table, possible when the index contains every column the query needs.

loops — in a query plan, how many times a node was executed. A node inside a nested loop runs once per outer row, so a cheap node with many loops can dominate a plan.

Partial index — an index covering only rows matching a condition. Smaller and cheaper, but usable only for queries whose conditions match.

Planner — the component that decides how to execute a query. It chooses between strategies by estimating cost, and the estimates depend on statistics.

Ring buffer — a small bounded allocation used for large sequential scans, so one big scan cannot evict everything else from the shared pool. Means such scans largely bypass that pool.

Sequential scan — reading a table from start to finish. Often the fastest option on a small table, regardless of what indexes exist.

shared_buffers — the memory pool Postgres keeps recently used pages in. Enlarging it does not help work that bypasses it.

Sharding, replication, pooling — none of these appear in this study. It examines a single database on a single connection.

Subtransaction — a transaction nested inside another, which can be rolled back without discarding the outer one. Used here so measurements leave no trace; the side effect is that some statistics counters are discarded on abort.

WAL (write-ahead log) — the durable record of every change, written before the change reaches the data files. Because its volume is determined by structure rather than timing, counting WAL records is immune to the scheduling noise that makes timings unreliable on a laptop.

work_mem — how much memory a single sort or hash operation may use before spilling to temporary files on disk.